Billing Widget UI Mock-Up Jones PropTech SaaS

Junior Automation Engineer Assignment – Almog Saadi

Part 2a – UI Problems & Issues Found

The following issues were identified by analyzing the billing widget mock-up across the dimensions of Security, Usability, and Functionality/Completeness.

Issue	Explanation	Impact	Functional Aspect
Missing CVV Field	The form collects Card Number, Expiration, and Cardholder Name, but has no field for the Card Verification Value (CVV) – the 3- or 4-digit security code printed on the card.	CVV is a mandatory data element in virtually payment. Without it, the transaction will be rejected at the processor level, meaning the form cannot complete a real payment in its current state. More critically, omitting CVV removes a key authentication layer that protects cardholders from fraudulent use of stolen card numbers.	Security
Plaintext Card Number Input – No Masking	The card number field appears to be a standard text input. There is no indication that the entered value is masked	Displaying the full card number in plaintext on screen exposes it to shoulder-surfing. It also	Security

	(displayed as hidden dots), auto-formatted, or tokenized client-side before transmission.	suggests the raw number may be transmitted and/or logged without proper tokenization, which is a PCI-DSS compliance violation.	
No HTTPS / Secure Transmission Indicator Visible	The mock-up does not surface any visual trust signal (padlock icon, "Secure Payment" badge, or references to encryption). While a live implementation may use TLS, the absence in the design means this was not considered.	Users are less likely to trust and complete the form. More critically, if the production implementation inherits the oversight, sensitive payment data could be transmitted over an unencrypted connection.	Security
Remove Dashes/Spaces from Card Number and Postal Code	The field label reads: "Card Number (No dashes or spaces)". Users are instructed to manually reformat their 16-digit card number.	This is an unnecessary cognitive burden. Card numbers are printed and mentally chunked in groups of four. Forcing users to strip separators increases transcription errors and form-abandonment rates. Industry best practice is	Usability

		to accept any reasonable format and sanitize server-side.	
Non-Editable Payment Amount	The "Payment Amount" is shown as a static value of 30.00 with no context. It cannot be changed by the user and there is no explanation of what it relates to.	A user arriving at this form without knowing they owe exactly 30.00 will be confused. The absence of currency, invoice context, or itemization undermines trust.	Functionality
MI Field	The Cardholder Name section includes a separate "MI" (Middle Initial) field, marked as optional.	Payment processors match the cardholder name against the name on the bank's record as a single string. Splitting first name, MI, and last name into three fields introduces a concatenation risk.	Usability
No Field-Level Validation Feedback Specified	The mock-up does not show any inline validation states.	Without clear, immediate validation feedback, users submitting incorrect data will only discover their mistake after a round-trip to the server.	Functionality

Part 2b – Sample Test Cases

TC-1: Happy Path – Successful Payment Submission

Test Case ID: TC-1

Title: Valid payment details submitted successfully

Type: Functional – Happy Path

Preconditions:

- User is authenticated and navigated to the billing payment screen.
- The form is in its default, empty state.
- A valid test Visa card is available.

Test Steps:

1. Select "VISA" from the Card Type dropdown. Expected: Dropdown updates to display "VISA".
2. Observe the Payment Amount field. Expected: Field displays 30.00 and is read-only.
3. Enter valid 16-digit number in the Card Number field. Expected: Number is accepted.
4. Enter 123 in the CVV field (field to be added). Expected: 3-digit value is accepted and masked.
5. Select "12" from the Expiration Month dropdown. Expected: Month is selected.
6. Select "2026" from the Expiration Year dropdown. Expected: Year is selected.
7. Enter "Alexandra" in the First Name field. Expected: Text is accepted.
8. Enter "B" in the MI field. Expected: Single character is accepted.
9. Enter "Bennett" in the Last Name field. Expected: Text is accepted.
10. Enter "123 Main Street" in the Billing Street Address field. Expected: Text is accepted.
11. Enter "San Francisco" in the City field. Expected: Text is accepted.
12. Select "CA" from the State/Province dropdown. Expected: State is selected.
13. Enter "94105" in the Postal Code field. Expected: 5-digit ZIP code is accepted.
14. Click the "Continue" button. Expected: Page navigates to a payment confirmation screen.

Expected Outcome: Payment is processed successfully. User sees a confirmation screen with a reference number and summary of the transaction.

TC-2: Negative Path – Missing Mandatory Fields

Test Case ID: TC-2

Title: Form submission blocked when all required fields are empty

Type: Functional – Negative Path

Preconditions:

- User is on the billing payment screen with all fields empty.

Test Steps:

1. Leave all form fields completely blank. Expected: All fields remain empty.
2. Click the "Continue" button. Expected: Form submission is blocked.
3. Observe the state of each required field. Expected: All required fields display a visible validation error.
4. Observe the error messages. Expected: Each error clearly describes what is required.
5. Observe page URL. Expected: URL remains on the payment form; no navigation occurs.

Expected Outcome: The form does not submit. All required fields surface distinct, clear, inline error messages. No network request is made to the payment processor.

TC-3: Negative Path – Invalid Card Number Format

Test Case ID: TC-3

Title: Invalid card number rejected with descriptive error

Type: Functional – Negative Path

Preconditions:

- User is on the billing payment screen.
- All other fields are filled with valid data.

Test Steps:

1. Enter "1234" (too short) in the Card Number field. Expected: Input is accepted by the field.
2. Click "Continue". Expected: Submission is blocked.
3. Observe the Card Number field. Expected: An inline error is shown.

4. Clear the field and enter a 16-digit number that fails Luhn check (a checksum formula for numbers/digits used with credit card or administrative numbers). Expected: Input is accepted by the field.
5. Click "Continue". Expected: Submission is blocked.
6. Observe the Card Number field. Expected: Inline error.
7. Enter a valid 16-digit number. Expected: Field accepts the value; error clears.
8. Click "Continue". Expected: Form proceeds to the next step.

Expected Outcome: Client-side validation rejects invalid card numbers immediately, without a server round-trip. Error messaging is specific and actionable.

Part 2c – Product Solution for the Most Severe Bug

Bug: Missing CVV Field

Severity: Critical.

This is the most severe issue because it simultaneously makes the form non-functional and insecure.

Recommended Solution

1. Add a CVV/CVC Input Field Insert a clearly labelled, required "CVV / Security Code" field immediately after the Card Number field. This ordering mirrors the physical layout of a payment card and matches the pattern users have learned from other checkout flows.

Field Specification:

- Label: CVV / Security Code
- Required: Yes
- Input type: password or masked text
- Max length: 4 characters
- Validation: Numeric only; 3 digits for Visa/Mastercard/Discover, 4 for Amex
- Helper text: "3 digits on the back of your card (4 digits for American Express)"

2. Implement Client-Side Masking The input should mask the value after entry to prevent shoulder-surfing.
3. Never Store the CVV Transmit the CVV only to the payment processor via their client-side SDK. The CVV must never be stored on Jones' own servers.
4. Adjust the Card Type Dropdown to Drive Validation When the user selects American Express, the CVV field label should automatically update to "CID (4 digits)" and its max length should change to 4.
5. Updated Acceptance Criteria (Definition of Done)

- CVV field is present, visible, and marked as required.
- CVV field accepts only numeric input.
- CVV field is masked (characters hidden after entry).
- For Visa/Mastercard/Discover: max 3 digits; for Amex: max 4 digits.
- CVV value is transmitted exclusively through the payment processor's secure SDK.
- CVV value is never logged, stored, or echoed back in any API response.
- Submitting with an empty CVV field surfaces an error.